The Visual Computer (2024) 40:3903–3917
https://doi.org/10.1007/s00371-024-03396-1

RESEARCH

Latent diffusion transformer for point cloud generation

Junzhong Ji[1,2] · Runfeng Zhao[1,2] · Minglong Lei[1,2]

Accepted: 29 March 2024 / Published online: 22 April 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract
Diffusion models have been successfully applied to point cloud generation tasks recently. The main notion is using a forward process to progressively add noises into point clouds and then use a reverse process to generate point clouds by denoising these noises. However, since point cloud data is high-dimensional and exhibits complex structures, it is challenging to adequately capture the surface distribution of point clouds. Moreover, point cloud generation methods often resort to sampling methods and local operations to extract features, which inevitably ignores the global structures and overall shapes of point clouds. To address these limitations, we propose a latent diffusion model based on Transformers for point cloud generation. Instead of directly building a diffusion process based on the points, we first propose a latent compressor to convert original point clouds into a set of latent tokens before feeding them into diffusion models. Converting point clouds as latent tokens not only improves expressiveness, but also exhibits better flexibility since they can adapt to various downstream tasks. We carefully design the latent compressor based on an attention-based auto-encoder architecture to capture global structures in point clouds. Then, we propose to use transformers as the backbones of the latent diffusion module to maintain global structures. The powerful feature extraction ability of transformers guarantees the high quality and smoothness of generated point clouds. Experiments show that our method achieves superior performance in both unconditional generation on ShapeNet and multi-modal point cloud completion on ShapeNet-ViPC. Our code and samples are publicly available at https://github.com/Negai-98/LDT.

Keywords 3D · Point cloud generation · Diffusion model · Transformers

1 Introduction

Point clouds contain a set of unordered points sampled from a 3D shape surface, which are widely used in many computer vision and graphical applications [25, 33, 47]. Point cloud generation is a basic task in this domain, aiming to learn the underlying shape distribution of 3D shapes and can serve as foundations for various downstream 3D conditional generative tasks, e.g., surface reconstruction [35, 48], point cloud completion [29, 45, 56], and view synthesis [6, 36].

✉ Minglong Lei
leiml@bjut.edu.cn

Junzhong Ji
jjz01@bjut.edu.cn

Runfeng Zhao
zhaottyxx@emails.bjut.edu.cn

1 Beijing Municipal Key Laboratory of Multimedia and Intelligent Software Technology, Beijing University of Technology, Beijing 100124, China

2 Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China

Deep learning has achieved great success in many computer vision tasks [8, 27], which includes different data structures such as images [7, 21], videos [40], and point clouds [32, 51]. In point cloud generation, early generative methods for point clouds treat the data as fixed-size matrices, which ignore the permutation invariance of point clouds. As a result, they fail to faithfully capture the underlying surface distribution [37, 41, 44]. Later, PointFlow [49] introduces the normalizing flow [12] into the point cloud generation domain, which builds a permutation-invariant network and treats points as an unordered set independently sampled from a shape distribution. It generated point clouds by maximizing the likelihood and is well-suited to the nature of unordered point clouds. After PointFlow, numerous probabilistic methods following permutation invariance for point cloud generation have been proposed. The mainstream existing works for point cloud generation are based on flow [22, 24], variational auto-encoder (VAE) [9, 23]. However, the intrinsic properties of flow and VAE, e.g., reversibility and variation, limit the expressiveness of their networks. Flow-based models suffer from restricted network design due to

the constraints of invertibility, which in turn leads to low expressiveness. VAE tends to over-smooth the latent vector distribution, leading to detail loss and hindering their ability to generate high-quality, complex samples.

Diffusion models [17] have achieved remarkable results in point cloud generation [2, 31, 53, 56]. The main idea is to use a forward process that adds noises into point clouds and a reverse process that iteratively denoises noisy samples using a denoising network. Different from VAE and flow, there is no clear network limitation on diffusion methods, which results in better expressiveness and broader mode coverage. The diffusion model can generate more complex, higher-quality, and more diverse point clouds [53].

However, current point cloud generation methods based on diffusion models still have several limitations. Firstly, the dimensional, unordered, and complex properties of point clouds make it difficult to directly apply diffusion models from other fields to effectively capture the surface distribution. Several works use external global encoding to guide point cloud generation, which limits the performance in other generation tasks and reduces its flexibility [2, 31, 53]. Secondly, current diffusion models are limited by sampling methods and local operations to capture the structures in point clouds. Therefore, these methods inevitably ignore the long-distance structure between points where the generated point clouds may lack smoothness.

To solve these limitations, we propose a novel transformer-based latent diffusion model for point cloud generation, called the latent diffusion transformer (LDT). We first employ a latent compressor to encode complex and irregular point clouds into easily learnable latent tokens. These tokens preserve the surface information of point clouds and follow a Gaussian distribution which has better fitness to diffusion models. Therefore, the difficulty of learning the surface distribution of point clouds can be reduced without external global encodings. Besides, the feature extraction capabilities of the diffusion model can also be enhanced. With tokenized representations, we can trivially achieve conditional generation by fusing the external information with cross-attention. Subsequently, we constructed a transformer-based diffusion generator on the tokenized representations. The strong capability of transformers to capture dependencies between tokens enables our diffusion generator to adequately capture global shapes and local details of point clouds. Therefore, our method can generate high-fidelity and smooth point clouds. Additionally, to effectively integrate transformers into the diffusion model, we introduce a dynamic positional encoding in the transformer, which transforms timesteps of diffusion into positional encodings and embeds the timestep information into the attention mechanism of the Transformer. Dynamic positional encoding strengthens the connection between diffusion model and diffusion process and effectively improves the ability to recover information from noisy samples.

Compared with previous methods, our advantages are threefold: (1) LDT does not require additional conditional assistance in the generation process to achieve high-quality generation, which enables our model to better adapt to complex conditional generation tasks. (2) LDT can effectively capture long-distance structural information and can generate smooth point clouds that effectively reflect the underlying surface distribution. (3) LDT has high expandability, as it is able to achieve superior generation performance in conditional generation tasks without task-specific architectures.

Our main contributions are as follows:

- We propose LDT, a novel latent diffusion model based on a transformer backbone for point cloud generation which is able to generate high-quality and smooth samples.
- We propose a method that applies transformers to diffusion models, providing a new transformer-based architecture in diffusion models for point cloud generation.
- Experiments show that LDT achieves superior generation and multi-modal completion performance on ShapeNet and ShapeNet-ViPC benchmarks.

2 Related works

2.1 Diffusion model for point clouds

Diffusion model has achieved remarkable success in various generative fields, e.g., image generation [11, 18, 38], video generation [16, 39], and speech generation [19, 20]. DDPM [17] first proposed the diffusion model and achieved excellent results in image generation. Although diffusion models can achieve high-quality generation with stable training, the training and generation of diffusion models often require huge computing resources, which limits further development. To solve this problem, stable diffusion [38] implements a denoise-based latent diffusion model that uses an auto-encoder to remove high-frequency details that affect generation and training efficiency. Stable diffusion significantly improves model efficiency and extensibility without compromising the generation quality and has achieved great success in the field of image generation and related tasks.

The generative ability of diffusion models depends on the fitness between models and data formats. Directly applying diffusion models from the above generation tasks to point cloud generation often results in poor performance. The format of point clouds is different from other types of data and a suitable network backbone for point cloud data needs to be designed. ShapeGF first introduces diffusion models for point cloud generation. It predicts the gradient directions and magnitudes (scores) in the gradient field using a score net-

work with an element-wise network (MLP) backbone and generates samples by Langevin dynamics sampling. To effectively guide the generation direction of the diffusion model, ShapeGF utilizes an additional conditional generation network that provides global structural information of the point cloud (i.e., shape latent) to assist in the score prediction. Following ShapeGF, most diffusion models for point cloud generation adopt global conditional generation networks to generate shape latent to guide the denoising network. DPM [31] extends the training and sampling methods of DDPM [17] to point cloud generation, which treats point cloud generation as a conditional generation process guided by the global shape latent. DPM proposes forward and reverse Markov chains based on the global shape latent to model forward and reverse diffusion processes. Similar to ShapeGF, DPM uses a global conditional generation network and an element-wise network backbone. The conditional generation network generates the shape latent to guide the diffusion process, while the backbone network predicts the noises introduced in the forward diffusion to achieve the sample generation.

The improvements of DPM and DDPM mainly lie in the design of the diffusion process. In contrast, PVD [56] focuses on the backbones of denoising in diffusion models. Instead of using the global shape latent to guide the generation process, PVD uses a point-voxel CNN (PVCNN) backbone to enhance the ability of feature extraction for point clouds. PVCNN voxelizes point clouds and aggregates neighborhoods to capture the local structure of point clouds, which addresses the limitation of element-wise networks in capturing local neighborhood information. Although PVD makes significant breakthroughs in generation performance, the high time complexity of neighborhood aggregation in PVCNN significantly decreases its generation efficiency, making the generation efficiency of PVD much lower than the aforementioned methods. Following PVD, LION [53] further introduces the latent diffusion model into point cloud generation. LION also adopts PVCNN as the backbone network and inherits the idea of designing an additional global shape latent. Due to the compression of geometric information in the latent states, LION can largely alleviate the difficulty to learn the distribution of point clouds. However, LION is much slower than PVD because of insufficient compression and the difficulty of scaling diffusion models.

In summary, on one hand, most methods use the global shape latent to guide the generation process. Although global shape latent has been proven to be effective in enhancing the expressive power of diffusion models, they are not conducive to the expandability of the model. On the other hand, current diffusion models use element-wise networks or point-voxel CNN (PVCNN) [30] as the neural network backbones, which struggle to capture long-range structural information.

2.2 Transformers for point clouds

Transformer has brought significant changes to the visual field [14, 28]. In image generation, vision transformers (ViTs) [13] have drawn widespread attention due to their powerful feature extraction capabilities. Many works combine ViTs with generative models to improve model expressiveness [5, 10, 34, 54]. DiT [34] first combines ViTs into the diffusion model. It strengthened the scalability and expression of the model by replacing the UNet architecture with the ViTs architecture in the denoising network and achieved SOTA generation performance in the image generation. Subsequently, a large number of researchers further developed the fusion of ViTs and diffusion models [3, 10]. Currently, ViTs have become one of the common backbone networks for diffusion models in the field of image generation.

In point cloud generation tasks [9, 23, 46, 51], transformers have also made great success due to their impressive abilities to capture the semantic information and modeling long-range dependencies in point clouds. SetVAE [23] interprets point cloud generation as the set generation and builds a hierarchical VAE with a set transformer [26]. PoinTr [51] builds geometry transformers for point cloud completion. The geometry-aware transformer takes advantage of the inductive bias of point clouds while simultaneously capturing the long- and short-range structural information of sets. SnowflakeNet [46] proposes a Snowflake point deconvolution (SPD) block based on skip-transformers. The dependencies between seed and child points can be adequately extracted by this framework. It can be noticed that transformer has been successfully used in the point cloud generation domain. However, it remains unexplored to insert transformer architectures into diffusion models to fully utilize their advantages.

3 Method

Point clouds are generally represented as a set of unordered points $X = \{x_i\}_{i=1}^{N}$ independently sampled from the surface distribution where $x_i \in \mathbb{R}^3$ and N represents the number of points. We propose a latent diffusion model for point cloud generation with a transformer neural backbone. To be concrete, our method contains a latent compressor and a diffusion generator. First, we represent the surface distribution into a Gaussian distribution represented by latent tokens through a latent compressor, which can reduce the difficulty of learning surface distribution for subsequent diffusion models. Subsequently, the diffusion generator with transformers builds the long- and short-range dependencies of tokens to effectively extract potential point cloud surfaces and generate high-quality and smooth samples. The overall model frame-

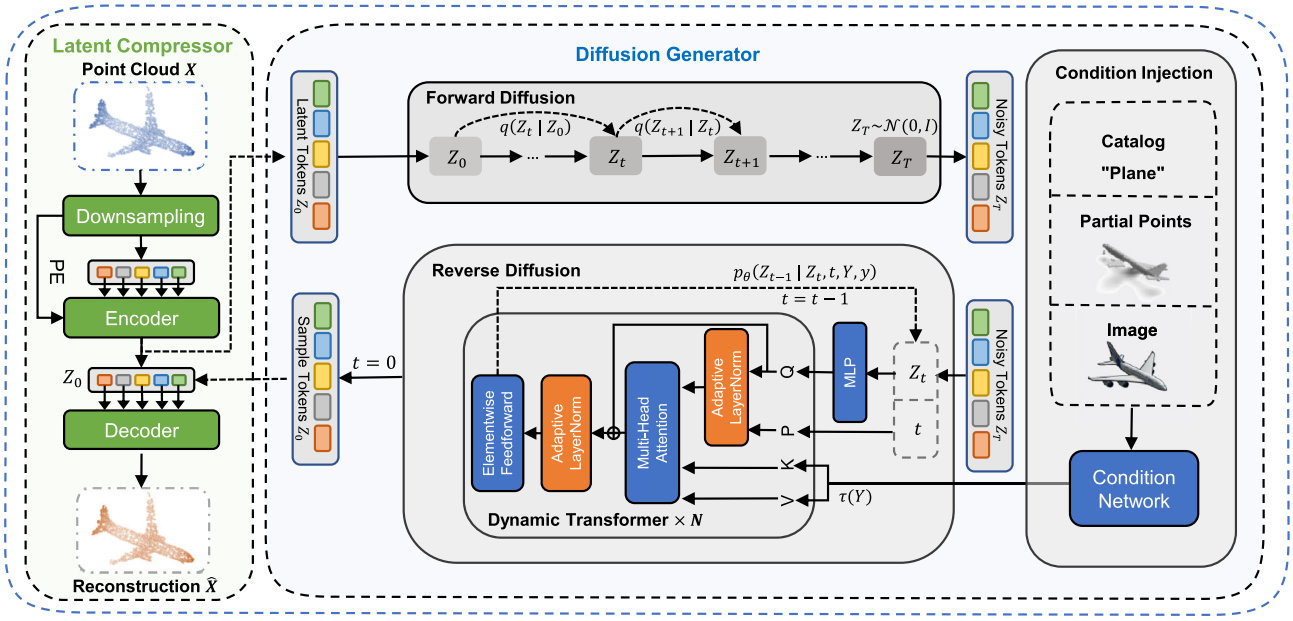

Fig. 1 Our method consists of a latent compressor and a diffusion generator. We tokenize the point cloud and represent it as small-size set-structure tokens in latent space with a trained latent compressor. Then, we generate latent tokens with the diffusion generator and convert them to point clouds with a decoder in the latent compressor

work is shown in Fig. 1. We first give the details of the two modules in our method and then introduce the conditional and unconditional generation processes based on our framework.

3.1 Latent compressor

To formulate a set of computational tokens that are suitable for the diffusion generator based on transformers, we map the original point clouds into the latent space where the tokens are high-level representations with high expressiveness and flexibility. To achieve this goal, we introduce a latent compressor that aims at compressing the complex surface distribution into a low-resolution Gaussian distribution. The latent compressor first downsamples point clouds into compact point sets to avoid computing overlarge relationship matrices. Subsequently, we utilize these point sets as inputs to an encoder–decoder to generate latent tokens.

3.1.1 Downsampling

It is necessary to convert the data into a series of subsets before passing through the feature extraction stage. In image processing [14], the image is first divided into multiple sub-images in a "zigzag" order from top left to bottom right. The patches are then aggregated to form a sequence of patches where each patch represents a local area of the image. Similarly, we borrow this idea and use the approach that processes images to partition point clouds. Unlike images, point clouds are set-structured data where the points are unordered. Specifically, we use the furthest point sample (FPS) to formulate several clustering centers where each center can represent a local area that contains a subset of points near the center. Then, we run the KNN algorithm to divide point clouds into several subsets of point clouds. Finally, we utilize an MLP to extract the features of points in each cluster and pool them into subset representations. Conceptually, the downsampling method can be formalized as follows:

$$\begin{aligned} C &= \mathrm{FPS}(X), \\ X_0 &= \mathrm{MaxPooling}\Big(\mathrm{MLP}(\mathrm{KNN}(X, C))\Big), \end{aligned} \tag{1}$$

where $C \in \mathbb{R}^{M\times 3}$ denotes the clustering centers, $X_0 \in \mathbb{R}^{M\times D}$ denotes the representations of subset point clouds, M represents the number of centers and subsets of points, and D is the dimension representations of subsets after downsampling.

3.1.2 Tokenization auto-encoder

After downsampling the point cloud, the latent compressor implements an attention-based auto-encoder to map the subset representations into a series of latent tokens $Z \in \mathbb{R}^{M\times D}$. Our encoder consists of a transformer with cluster centers as positional encodings. On the one hand, we incorporate the geometric information of point clouds into the tokens

through positional encodings, ensuring that the tokens effectively capture local information from different regions of point clouds. On the other hand, the self-attention mechanism is leveraged to depict the dependencies among the tokens, which effectively capture the global information from point clouds. By this means, we integrate most of the information of point clouds into the tokens with the encoder, ensuring the effectiveness of subsequent reconstructions.

Our decoder consists of a transformer with cross-attention. Similar to SetVAE [23], we learn a seed $h \in \mathbb{R}^{N' \times D}$ as the query of the decoder's cross-attention. We use cross-attention to fuse the information of latent tokens Z into the seed and map it to the point cloud through an MLP to achieve point cloud reconstruction from latent tokens. By sampling queries with different sizes from the seed, we can reconstruct point clouds with different resolutions. Although our method does not explicitly preserve the geometric information in the latent space, we still have the capacity to effectively learn the local information of point clouds as each token represents a local area of the point clouds.

3.2 Diffusion generator

The diffusion generator is constructed in the latent space of the tokenization auto-encoder, which consists of forward and reverse diffusion processes. The key part of the diffusion generator is a denoising network based on transformers. The forward process is utilized to train the denoising network and the reverse process utilizes the denoising network for generation.

3.2.1 Forward diffusion

Forward diffusion is constructed as a Markov chain and progressively adds noises to samples. Denote the tokens generated by the latent compressor as Z_0, the forward process starts from the latent distribution $q(Z_0)$ and ends at a normal distribution $q(Z_T)$:

$$\begin{aligned} q(Z_{1:T}|Z_0) &= \prod_{t=1}^{T} q(Z_t|Z_{t-1}) \\ &= \mathcal{N}(Z_t; \sqrt{\alpha_t} Z_0, 1 - \alpha_t \mathbf{I}), \end{aligned} \tag{2}$$

where t is the timestep, $\alpha_t = \prod_{i=1}^{t}(1-\beta_t)$, β_t denotes the variance schedule hyper-parameter. By using the reparameterization trick, Z_t can be sampled from $Z_t = \sqrt{\alpha_t} Z_0 + \sqrt{1-\alpha_t}\epsilon_t$, where $\epsilon_t \sim \mathcal{N}(0, \mathbf{I})$. By utilizing the denoise network $\epsilon_\theta(Z_t, t)$ that predicts the noise ϵ introduced in the forward diffusion, we can then build the reverse diffusion to generate the latent states.

3.2.2 Reverse diffusion

The reverse diffusion gradually removes noises from the noisy samples, which is also the generation process of the diffusion model. The reverse diffusion process can be expressed as follows:

$$\begin{aligned} &p_\theta(Z_0|Z_T) \\ &\quad = p(Z_T) \prod_{t=1}^{T} p_\theta(Z_{t-1}|Z_t) \\ &\quad = p(Z_T) \prod_{t=1}^{T} \mathcal{N}\left(Z_t - \frac{\beta_t}{\sigma_t \sqrt{1-\beta_t}} \epsilon_\theta(Z_t, t), \sigma_t^2 \mathbf{I}\right), \end{aligned} \tag{3}$$

where θ represent the parameters of the diffusion generator, $p(Z_T)$ is the standard normal distribution, σ_t is the variance hyper-parameter, and $p_\theta(Z_0)$ is the generated latent states. To simplify the design of the diffusion model, σ_t is designed as a hyper-parameter. Our diffusion generator learns the reverse diffusion with a permutation-invariant network $\epsilon_\theta(Z_t, t)$ based on transformers. Starting from a normal distribution $p(Z_T)$, the network iteratively predicts and denoises the noise ϵ introduced in the forward diffusion, thereby extracting clean sample distributions $p_\theta(Z_0|Z_T)$ from the noisy sample distributions. We can generate latent states by gradually denoising the noisy samples from a normal distribution.

3.2.3 Dynamic transformer

The denoising network plays an essential role in the diffusion process. To construct a denoising network that can effectively model long-range dependencies, we propose the dynamic transformer as the neural backbone. The transformer is capable of modeling long-range dependencies and capturing long-distance structures through self-attention.

Additionally, to enhance the ability to recover clean samples from high-noise samples, we introduce a dynamic position encoding to incorporate temporal information of noisy samples. We encode the position encoding of the samples into the means and variances with MLPs and dynamically adjust the mean and variance of each layer. For the unconditional generation, the dynamic transformer is defined as

$$\begin{aligned} &Q_t^{(l)} = Z_t^{(l)} W_Q^{(l)}, K_t^{(l)} = Z_t^{(l)} W_K^{(l)}, V_t^{(l)} = Z_t^{(l)} W_V^{(l)}, \\ &Z_t^{(l+1)} = \mathrm{AdaLN}\left(\mathrm{MLP}(\mathrm{softmax}\left(\frac{Q_t^{(l)}(K_t^{(l)})^\top}{\sqrt{D^{(l)}}}\right) V_t^{(l)}), P_t\right), \end{aligned} \tag{4}$$

where $Z_t^{(l)}$ is the embedding of latent tokens at the l-th layer, $D^{(l)}$ is the dimensions of features, P is the position encoding which is determined by the time positional encoding of t and $\mathrm{AdaLN}(\cdot, P)$ is the adaptive layer normalization.

Algorithm 1 Algorithm of Unconditional Generation

First Stage Training
Input: Data X, Parameters $\{\xi, \psi\}$
Output: Parameters $\{\xi, \psi\}$
while Not Converged **do**
  Sample $X \sim p_{data}(X)$
  Sample $Z \sim q_\xi(Z|X)$
  Reconstruct $\hat{X} \sim p_\psi(\hat{X}|Z)$
  Compute $\mathcal{L}(\xi, \psi) \leftarrow Eq.(5)$ propagate gradient
  Update $\{\xi, \psi\}$
end while
Second Stage Training
Input: Data X, Parameters $\{\xi, \psi, \theta\}$
Output: Parameters $\{\xi, \psi, \theta\}$
while Not Converged **do**
  Sample $X \sim p_{data}(X)$
  Sample $Z \sim q_\xi(Z|X)$
  Reconstruct $\hat{X} \sim p_\psi(\hat{X}|Z)$
  Sample $t \sim Uniform(0, \mathbf{I})$
  Sample $Z_t \leftarrow Eq.(2)$
  Compute $\mathcal{L}(\theta, \xi, \psi) \leftarrow Eq.(6)$ propagate gradient
  Update $\{\theta, \xi, \psi\}$ or $\{\theta\}$
end while
Unconditional Generation
Input: Parameters $\{\psi, \theta\}$, Steps T
Output: Samples $\hat{X}$
Sample $Z_T \sim N(0, I)$
Sample $Z_0 \leftarrow Eq.(3)$
Reconstruct $\hat{X} \sim p_\psi(\hat{X}|Z_0)$

The AdaLN layer uses an MLP to dynamically adjust the mean and variance of each layer according to the positional encoding. In other words, the means and variances inserted in each layer are generated from the position encoding P through an MLP. In the setting of the conditional generation where external conditions (e.g., images or incomplete point clouds) are available, $K_t^{(l)}$ and $V_t^{(l)}$ are calculated from external conditions through a conditional network, i.e., $K_t^{(l)} = \tau(Y)W_K^{(l)}$, $V_t^{(l)} = \tau(Y)W_V^{(l)}$, where τ is the condition network and Y is the external conditions.

3.3 Generation process

Our generative model is a general method that can achieve both unconditional generation and conditional generation. In this subsection, we will introduce these two generation processes in sequence. Notice that the conditional generation process requires external conditions, i.e., images or incomplete point clouds, it has often been used for point cloud completion.

3.3.1 Unconditional generation

In the first part, we will introduce the training and generation process for unconditional generation. The training and generation process of unconditional generation is summarized in Algorithm 1. First, we utilize a two-phase scheme in the training process. We first train the latent compressor to learn the latent tokens for point clouds in the first phase. Then, we train the diffusion generator for point cloud generation in the second phase.

The training objective of the first phase minimizes the reconstruction errors of the latent compressor, which extracts the high-level information in point clouds. The loss function to be optimized is:

$$\begin{aligned}\mathcal{L}(\xi, \psi) = \mathbb{E}_{q_\xi(Z|X)}[-\log p_\psi(X|Z)] \\ - \lambda_{\mathrm{KL}} \cdot D_{\mathrm{KL}}(q_\xi(Z|X)||p(Z)),\end{aligned} \tag{5}$$

where ξ, ψ are the parameters of encoder and decoder in the latent compressor, respectively, $q_\xi(Z|X)$ is the distribution of the latent tokens in encoder, $p_\psi(X|Z)$ is the distribution of the reconstructed point cloud in decoder, $p(Z)$ is a standard normal distribution, and λ_{KL} are hyper-parameters used to control the weight of KL divergence. Specifically, $\mathbb{E}_{q_\xi(Z|X)}[-\log p_\psi(X|Z)]$ is the reconstruction term implemented by computed the Chamfer distance (CD) and Earth mover's distance (EMD) between reconstruction and ground truth, and $D_{\mathrm{KL}}(q_\xi(Z|X)||p(Z))$ is a penalty term of KL divergence between the distribution of the latent tokens and the standard Gaussian distribution.

The second phase of training aims to maximize the log-likelihood of the point clouds $\log p_{\theta,\psi}(X|Z_T)$. We turn to minimizing the variational upper bound with the help of variational inference. Following the previous work [43], we ignore these constants that are not relevant to the training. The loss function is formulated as follows:

$$\begin{aligned}\mathcal{L}(\theta, \xi, \psi) = \mathbb{E}_{q_\xi(Z|X)}[-\log p_\psi(X|Z)] \\ + \mathbb{E}_{q_\xi(Z|X)}\Big[\log q_\xi(Z|X) \\ + \mathbb{E}_{q(Z_t|Z)}\big[w_t||\epsilon - \epsilon_\theta(Z_t, t)||_2^2\big]\Big],\end{aligned} \tag{6}$$

where θ is the parameter of diffsion generator, $\epsilon_\theta(\cdot, \cdot)$ is the denoise network, ϵ is the noise in Z_t, and w_t is the hyper-parameter utilized to control the weights of the latent compressor and diffusion generator. Specifically, $\mathbb{E}_{q_\xi(Z|X)}\Big[\log q_\xi(Z|X)]$ is the entropy of the distribtuion of latent tokens.

According to the above loss function Eq. (6), the second phase of training is divided into two cases depending on whether the compressor is fixed. When the latent compressor is fixed, the mapping from point clouds to latent states remains unchanged. The training process that maximizes the likelihood of the latent tokens can be regarded as learning the distribution of point clouds. When the latent compressor is unfixed, the training of the hybrid model can be regarded as optimizing a variational latent compressor with a prior distribution modeled by the diffusion generator.

Algorithm 2 Algorithm of Conditional Generation

Diffusion Generator Retraining
Input: Data X, Conditions Y, Parameters $\{\xi, \psi, \theta\}$
Output: Parameters $\{\xi, \psi, \theta\}$
while Not Converged **do**
  Sample $X \sim p_{data}(X)$
  Sample $Z \sim q_\xi(Z|X)$
  Reconstruct $\hat{X} \sim p_\psi(\hat{X}|Z)$
  Sample $t \sim Uniform(0, \mathbf{I})$
  Sample $Z_t \leftarrow Eq.(2)$
  Compute $\mathcal{L}(\theta, \xi, \psi, Y) \leftarrow Eq.(7)$ propagate gradient
  Update $\{\theta, \xi, \psi\}$ or $\{\theta\}$
end while
Conditional Generation
Input: Parameters $\{\psi, \theta\}$, Steps T, Conditions Y
Output: Samples $\hat{X}$
Sample $Z_T \sim N(0, \mathbf{I})$
Embedding Condition $\tau_\theta(Y)$
Sample $Z_0 \leftarrow Eq.(8)$
Reconstruct $\hat{X} \sim p_\psi(\hat{X}|Z_0)$

Second, our unconditional generation process can be divided into two parts: generating the latent states by the diffusion generator and decoding latent states as point clouds by the decoder of the latent compressor. Specifically, the process begins with a Gaussian noise $p(Z_T)$ and then undergoes the reverse diffusion to generate the latent states $p_\theta(Z_0|Z_T)$. Next, we sample any number of points from the seed h of the decoder as the query of cross-attention. The query is then fed into the decoder of the latent compressor. We use cross-attention to integrate the semantic information of the latent states Z_0 so that the point clouds $p_{\theta,\psi}(X|Z_T)$ can be reconstructed from the latent states.

3.3.2 Conditional generation

In this section, we introduce how to inject conditions into unconditional generation to achieve conditional generation. The external conditions can be labels, incomplete point clouds, and data from other modalities. The training and generation process of conditional generation is summarized in Algorithm 2.

To enhance the extensibility of the model and adapt it to various generation tasks, we design a conditional injection method on the dynamic Transformer. We first preprocess the external conditions and convert them into representations that are suitable for the dynamic Transformer. We design a conditional network $\tau_\theta(\cdot)$ to learn the embeddings of external conditions. Since the data formats of conditions are different for different conditional generation tasks, the architectures of conditional networks are also different. For simplicity, we use Y to denote any condition that can be added to our framework and use τ_θ to denote the corresponding conditional network. After embedding the condition into a suitable representation $\tau_\theta(Y)$ for the transformer, we fuse the semantic information of the condition through the cross-attention mechanism in the dynamic transformer. Then, we retrain the diffusion generator where the training process is similar to the unconditional generation with only small changes in the training objective. The loss function is formulated as follows:

$$\begin{aligned}\mathcal{L}(\theta, \xi, \psi, Y) = \mathbb{E}_{q_\xi(Z|X)}[-\log p_\psi(X|Z)] \\ + \mathbb{E}_{q_\xi(Z|X)}\Big[\log q_\xi(Z|X) \\ + \mathbb{E}_{q(Z_t|Z)}\big[w_t||\epsilon - \epsilon_\theta(Z_t, t, \tau(Y))||_2^2\big]\Big],\end{aligned} \tag{7}$$

where Y is the external condition and τ is the condition network.

After retraining the diffusion generator, we can achieve specific conditional generation tasks. The conditional generation process follows the same steps as the unconditional generation process where only the reverse diffusion is different. We employ a classifier-free generation method that predicts the noise that combines the results of unconditional and conditional denoising networks. The reverse process can be formulated as follows:

$$\begin{aligned}p_\theta(Z_{t-1}|Z_t, Y, \hat{Y}) = \mathcal{N}(Z_t - \frac{\beta_t}{\sigma_t\sqrt{1-\beta_t}}(\hat{\alpha}\epsilon_\theta(Z_t, t) \\ + \hat{\beta}\epsilon_\theta(Z_t, t, \tau(Y))), \sigma_t^2\mathbf{I}),\end{aligned} \tag{8}$$

where $\hat{\alpha}$ and $\hat{\beta}$ are hyper-parameters.

4 Experiments

We carry out experiments of unconditional generation and conditional generation to verify the effectiveness of our method. In the unconditional generation setting, we compare the generation performance and efficiency with baseline methods. Additionally, we conduct experiments of high-resolution point cloud generation and shape interpolation to prove the flexibility of our method. In the conditional generation section, we perform class-conditional generation and multi-modal point cloud completion based on external conditions to demonstrate the effectiveness of our method.

4.1 Experimental settings

4.1.1 Datesets

For unconditional point cloud generation, we utilize the ShapeNet [4] dataset provided in PointFlow [49] which includes 55 categories of shapes, 51,127 point clouds, and 15,000 points for each point cloud. We focus on Airplane, Chair, and Car as our primary datasets for unconditional generation following previous works [2, 23, 31, 56]. For point cloud completion that uses additional conditional networks with partial points and renders images as conditions,

we use the ShapeNet-ViPC dataset [55] that contains 38,328 objects across 13 categories. Each object includes 24 viewpoints of point clouds that are sampled from the ShapeNet uniform mesh. Partial point clouds are generated from the corresponding viewpoints, along with corresponding image renderings from the viewpoint with a size of 224×224. The image rendering is accomplished by using the configuration in ShapeNetRendering [4]. Following previous works [1, 57], we use a pair of incomplete point clouds and rendered images as external conditions, where the images are randomly selected from arbitrary viewpoints in both training and inference stages. For both generation and completion tasks, we use 80% of samples for training and the others 20% of samples for testing. We normalize all point clouds with a bounding sphere with a radius of 1.

4.1.2 Evaluation metrics

In unconditional generation, we utilize Chamfer distance (CD) and Earth mover's distance (EMD) as distance metrics to measure generation quality. Specifically, we calculate the minimum matching distance (MMD) and 1-nearest neighbor accuracy (1-NNA) using the aforementioned distance metrics. The MMD score represents the minimum matching distance between the samples and ground-truth point clouds, which reflects the quality of the samples. The 1-NNA evaluates the classification accuracy between the generated and ground truth by the 1-nearest neighbor classifier. It represents the distance between the distributions of generated points and ground truth, reflecting the diversity and fidelity of generated samples. For point cloud completion, we utilize Chamfer distance (CD) and F-score as the performance evaluation metrics.

4.1.3 Implementation details

In our experiments, the feature dimension for the latent compressor is set to 128. For downsampling, the size of clustering centers is 32 and the number of nearest neighbors of KNN is 128. The encoder and decoder contain six self-attention layers and six cross-attention layers, respectively. The dimensions of latent tokens from the latent compressor are 32×120 for each point cloud. For the diffusion generator, the feature dimension is 1024. In the setting of unconditional generation, we use a transformer with 24 layers of self-attention. In the setting of conditioned generation, we use a transformer with a combination of 12 layers of self-attention and 12 layers of cross-attention.

In the first phase of training, the learning rate of the latent compressor is 1×10^{-2}, and the weight of KL divergence is $\lambda_{KL} = 10^{-5}$. This phase involves 4000 epochs of training. In the second phase of training, we utilize the exponential moving average (EMA) to ensure training stability, where EMA decay is 0.9999 and the learning rate is 1×10^{-3}. This phase performs 5000 epochs of training. For conditional generation tasks, we train 1000 epochs for each dataset. For the setting where the latent compressor is not fixed, we train the second phase and then finetune the latent compressor for additional 1500 epochs at a learning rate of 1×10^{-3}. The experiments are conducted on an NVIDIA RTX 3090 GPU. The entire training process on the airplane dataset for unconditional generation takes approximately 2 days.

4.2 Unconditional generation

4.2.1 Comparison with the state-of-the-arts

We report the results of unconditional generation in this subsection where no external information is provided during the generation. We quantitatively compare our unconditional

Table 1 Comparison of generation performance

Category	Model	MMD (↓)		1-NNA (%↓)	
		CD	EMD	CD	EMD
Airplane	ShapeGF	1.02	8.00	80.62	88.02
	PointFlow	1.15	6.31	85.80	83.09
	SetVAE	1.04	6.16	89.51	87.65
	DPM	1.10	7.11	86.67	90.49
	PVD	1.12	6.17	80.25	77.65
	CanonicalVAE	0.83	5.50	63.45	71.60
	LION	1.02	5.79	71.11	70.86
	Ours	**0.80**	**5.18**	**56.17**	**63.21**
	Training set	1.01	5.72	69.15	64.20
Chair	ShapeGF	7.17	11.85	61.78	64.27
	PointFlow	7.26	12.12	65.56	65.79
	SetVAE	7.60	12.10	65.79	70.39
	DPM	**6.81**	11.91	64.65	69.26
	PVD	7.65	11.87	60.05	59.52
	CanonicalVAE	7.37	11.75	60.12	61.93
	LION	7.69	11.77	**55.89**	59.14
	Ours	7.01	**11.15**	59.67	**58.76**
	Training set	7.84	11.86	57.79	54.15
Car	ShapeGF	3.63	9.11	60.09	61.36
	PointFlow	3.69	9.03	63.78	57.67
	SetVAE	3.63	9.05	65.91	67.61
	DPM	3.70	9.39	74.01	73.15
	PVD	3.74	9.31	65.62	63.35
	CanonicalVAE	3.31	8.89	55.68	57.81
	LION	3.71	8.99	56.53	**52.84**
	Ours	**3.21**	**8.63**	**53.35**	53.13
	Training set	3.83	9.22	48.72	51.13

MMD-CD and MMD-EMD is scaled by 10^3 and 10^2 individually
Bold indicate the best results

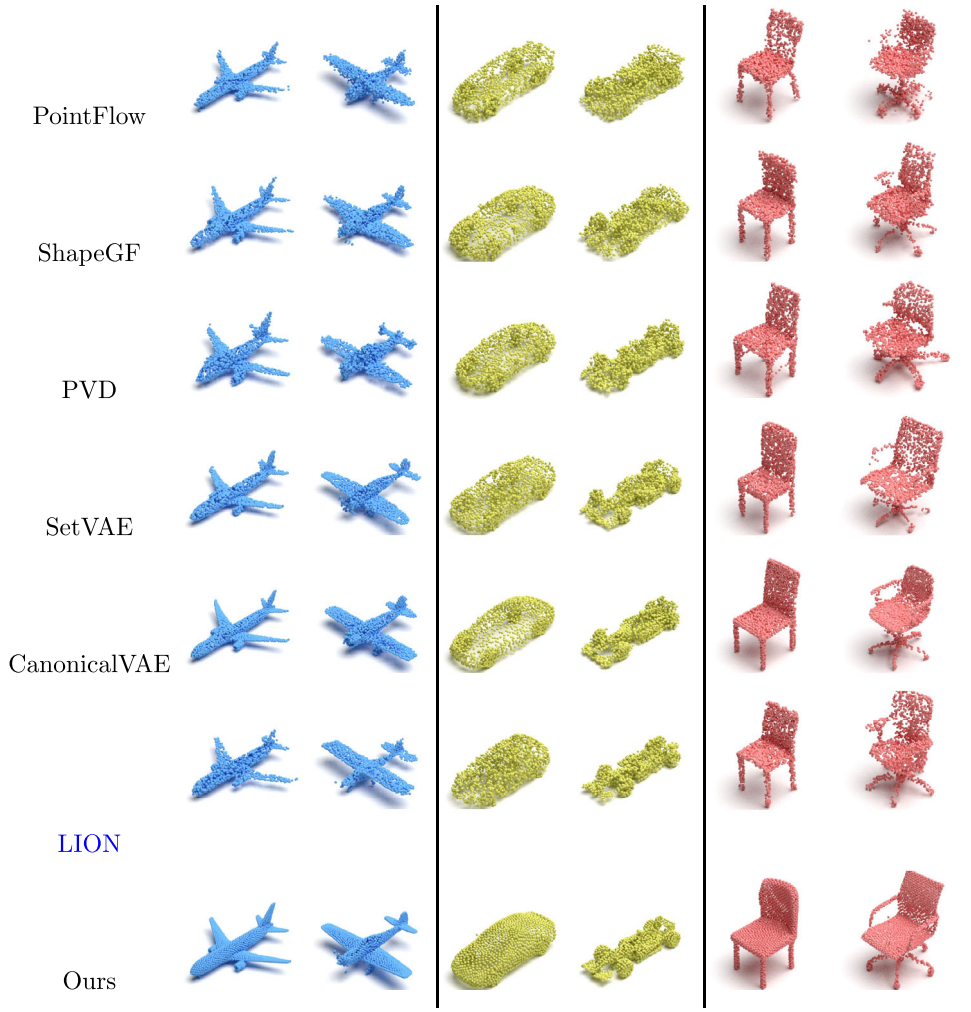

Fig. 2 Visualization results of unconditional point cloud generation on ShapeNet with 2,048 points

generation performance against several SOTA methods and present the results in Table 1. It shows that LDT outperforms previous models on all datasets. The performance advantages are particularly obvious in the car and airplane datasets. Compared with the SOTA latent diffusion model LION, our method demonstrates significant advantages in MMD, indicating that the point clouds generated by LDT have better uniformity. Meanwhile, our method also has surpassed performance on the 1-NNA metric on the relatively airplane dataset, which indicates that LDT can more accurately learn the distribution of complex point clouds. We also provide visual comparisons in Fig. 2 to intuitively demonstrate the different qualities of generation. As shown in Fig. 2, our generated samples exhibit remarkable uniformity with minimal outliers. This can be attributed, in part, to the transformer's ability to quantify the relationships between points, accurately measure inter-point distances, and capture the global shape of the point cloud. These capabilities ensure the LDT accurately captures the overall surface distribution, leading to superior generation performance. However, LDT exhibits limitations on the chair dataset. This might be attributed to the Transformer's tendency to prioritize low-frequency information, such as low-curvature regions in the point cloud. This prioritization can lead to a loss of high-frequency details in the latent tokens, consequently causing the model to favor generating point clouds with lower curvature. As chair point clouds typically exhibit higher curvature, this bias results in suboptimal generation performance for chair.

Then, we quantitatively compare the performance of LDT against PVD [56], a diffusion-based method for point cloud generation, at different numbers of function evaluations with the sampling method in DDPM [17]. As shown in Fig. 3, our method achieves comparable performance with PVD in only 25 function evaluations and reached optimal performance at around 300 function evaluations, while PVD required approximately 1000 function evaluations to achieve high-quality generation. Our method has a preferable inference efficiency that takes only 0.28 s per sample, which is nearly 20 times faster than PVD's 5.56 s and 100 times faster than LION's 27.12 s. The results are attributed to the suitable computation space of latent states and the better adaptation of the backbone to point clouds.

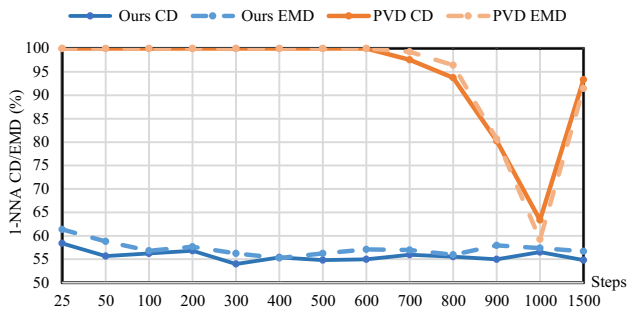

Fig. 3 Generation performance of our method and PVD [56] under different numbers of functions evaluations on the Car dataset

4.2.2 High-resolution point cloud generation

Our method can generate high-resolution samples by training with an initial state with a larger seed size. We train LDT with a maximum resolution of 10, 000 points and visualize the samples with different resolutions in Fig. 4. It can be seen that LDT is able to generate high-resolution point clouds. Our generated samples are smoothly distributed on the surface of the shape, indicating that our method effectively captures the surface distribution of point clouds.

4.2.3 Shape interpolation

Our method can achieve superior shape interpolation results that aim to merge different shapes of point clouds. By passing through two shapes into the latent compressor, we can obtain fused representations that contain the information from the two shapes. The results of interpolation are visualized in Fig. 5. Two original shapes are located on the leftmost and the rightmost sides of the figure, and others are the results of interpolations. It can be observed that the interpolations retain the characteristics of both generated point clouds and exhibit a morphing effect, which shows that the latent tokens capture the intrinsic information in point clouds and support removing and merging substructures of the shapes.

4.3 Conditional generation

In this subsection, we present the results of conditional generation. In general, the conditions used for generation can be labels or data from other modalities, i.e., images. We separately conducted the class-conditional generation and multi-modal point cloud completion to demonstrate the flexibility and effectiveness of our method.

4.3.1 Class-conditional generation

Class-conditional generation on ShapeNet aims to cover all categories of shape distributions and is required to sample from the corresponding category shape distribution with a category condition. We employ the conditional mechanism and use the label information as part of the position encoding to achieve class-conditional generation on the 55 categories of ShapeNet. We evaluate the generated samples of Airplane, Chair, and Car in Table 2. For simplicity, 20 categories of samples are selected for visualization, which is shown in Fig. 6. Although the performance decreases significantly compared to unconditional generation, our method can still generate high-quality point cloud samples.

4.3.2 Multi-modal point cloud completion

We employ the conditional mechanism with partial point clouds and rendered images as external conditions to achieve multi-modal point cloud completion on the ShapeNet-ViPC dataset. To alleviate the impact of multiple possible output on the completion results, for each incomplete point cloud, we perform 24 completions based on its rendered images from 24 viewpoints, and then take the average performance to compare with other methods. We conduct class-specific training and quantitatively compare the completion performance of our method with the SOTA point cloud completion methods in Tables 3 and 4. We also provide visual comparisons with other completion methods in Fig. 7. It shows that

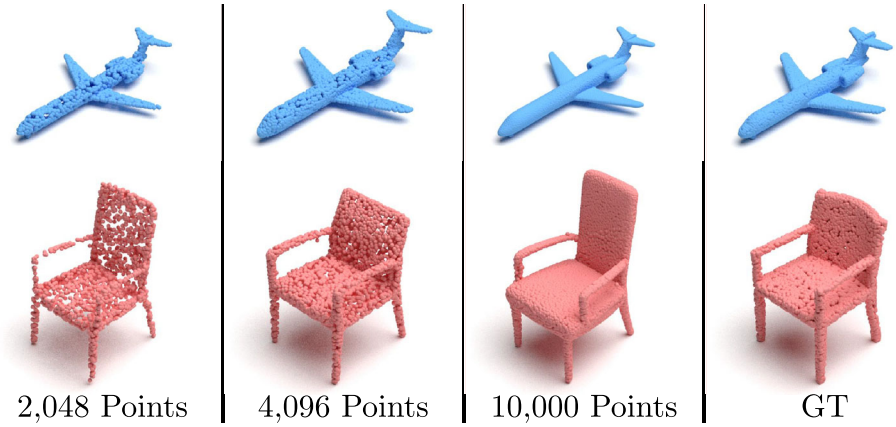

Fig. 4 Results of unconditional generation at various resolutions. The model is trained on a maximum resolution of 10,000 points

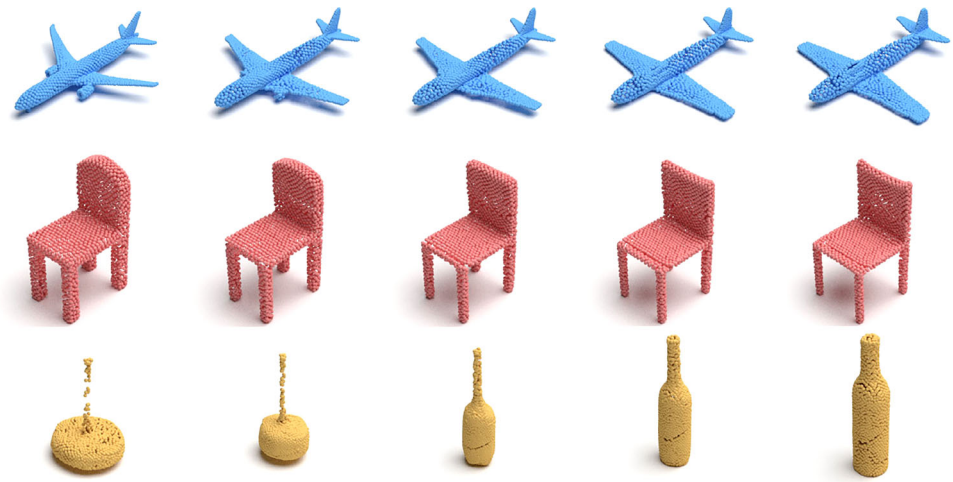

Fig. 5 Visualization of latent state interpolation between different generated point clouds with 2,048 points. The top is a plane-to-plane interpolation, the middle is a chair-to-chair interpolation, and the bottom is a lamp-to-bottle interpolation

Table 2 Generation performance on class-conditional generation

Dataset	MMD(↓)		1-NNA(%↓)	
	CD	EMD	CD	EMD
Airplane	0.91	5.80	79.84	76.91
Chair	7.13	11.53	57.76	62.30
Car	3.52	9.08	61.68	62.36

MMD-CD and MMD-EMD are scaled by 10^3 and 10^2 individually

our generation model has superior performance among all SOTA methods on both metrics without task-specific architectures. Our high completion performance stems from our superior generative ability and the good fitness between the diffusion generator and the conditional mechanism. These results prove the flexibility and effectiveness of our conditional mechanism and the high extensibility of our method.

4.4 Ablation study

We give an ablation study to analyze the impact of fixed and unfixed latent compressors. Firstly, we train the diffusion model with a fixed latent compressor and then finetune the model without fixing the latent compressor. We compare the generation performance of fixed and unfixed settings in Table 5. The result shows that the generation performance is better with an unfixed latent compressor. There are gaps between the sample distribution $p_\theta(Z_0|Z_T)$ and the latent space distribution $p_\xi(Z_0|X)$, and also between the reconstruction distribution $p_{\theta,\psi}(X|Z_T)$ and the point cloud distribution $p(X)$. We assume that optimizing both the latent compressor and the diffusion generator is equivalent to reweighting the two gaps, which leads to better results compared with only optimizing the first gap.

To prove the effectiveness of the dynamic transformer, we conduct a quantitative comparison between the performance of the model with a dynamic transformer backbone and the model with the vanilla transformer [26] in point cloud generation to demonstrate the effectiveness of the dynamic position encoding. As shown in Table 6, the one with the dynamic transformer backbone outperforms the vanilla transformer in both MMD and 1-nearest-neighbor metric. This proves that dynamic position encoding can effectively improve the expressiveness of the diffusion generator.

We also conducted ablation experiments to find impact of different reconstruction losses on the performance of the

Fig. 6 Results of class-conditional generation for 21 categories on the ShapeNet dataset

Table 3 Comparison of completion performance on ShapeNet-ViPC using CD (the lower, the better) with 2,048 points

Methods	Avg	Sofa	Chair	Cabinet	Lamp	Car	Watercraft	Table	Airplane
PCN [52]	5.619	5.668	7.441	6.331	4.840	6.409	3.510	6.508	4.246
FoldingNet [50]	6.271	6.368	8.823	6.958	6.504	5.307	3.882	7.080	5.242
ViPC [55]	3.308	4.481	2.476	4.558	2.867	3.183	2.197	4.990	1.760
AtlasNet [15]	6.062	6.023	8.161	6.414	7.182	4.868	4.261	6.561	5.032
TopNet [42]	4.976	5.281	6.391	5.629	5.547	4.530	3.350	5.381	3.710
CSDN [57]	2.570	3.240	2.835	3.670	2.554	2.977	1.742	2.575	1.251
PoinTr [51]	2.851	3.507	3.111	4.001	2.928	3.203	1.737	2.845	1.686
XMFnet [1]	1.443	1.702	1.403	1.980	1.810	1.754	0.945	1.386	0.572
Ours	1.184	1.349	1.176	1.716	0.982	1.730	0.787	1.222	0.513

CD is scaled by 10^3

Table 4 Comparison of completion performance on ShapeNet-ViPC using F-score (the higher, the better) with 2,048 points

Methods	Avg	Sofa	Chair	Cabinet	Lamp	Car	Watercraft	Table	Airplane
PCN [52]	0.407	0.293	0.323	0.456	0.331	0.270	0.577	0.431	0.578
FoldingNet [50]	0.331	0.249	0.204	0.237	0.360	0.300	0.518	0.351	0.432
ViPC [55]	0.591	0.434	0.529	0.706	0.512	0.451	0.730	0.594	0.803
AtlasNet [15]	0.410	0.318	0.379	0.326	0.426	0.304	0.551	0.469	0.509
TopNet [42]	0.467	0.361	0.388	0.358	0.491	0.405	0.615	0.528	0.593
CSDN [57]	0.695	0.557	0.669	0.548	0.761	0.560	0.782	0.729	0.862
PoinTr [51]	0.683	0.547	0.662	0.516	0.742	0.545	0.780	0.723	0.842
XMFnet [1]	0.796	0.723	0.809	0.662	0.792	0.691	0.901	0.830	0.961
Ours	0.854	0.804	0.858	0.733	0.909	0.729	0.939	0.875	0.982

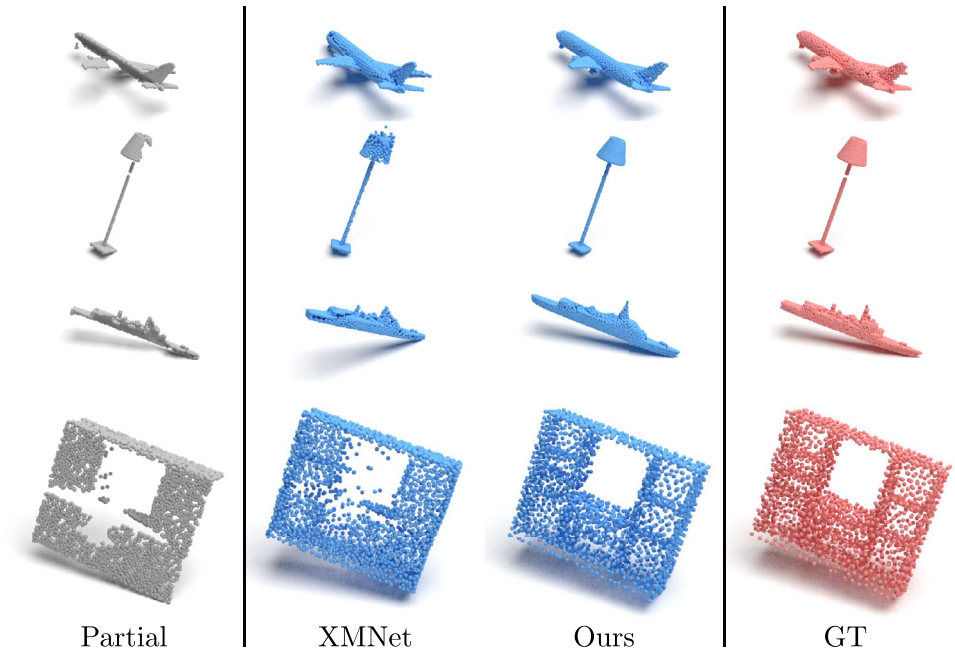

Fig. 7 Results of point cloud completion for airplane, lamp, watercraft, and cabinet with 2,048 points

Table 5 Comparison of generation performance between different training algorithms of fixed and unfixed auto-encoder in Sect. 3.3.1

Dataset	Model	MMD(↓)		1-NNA (%↓)	
		CD	EMD	CD	EMD
Airplane	Fixed	0.83	**5.14**	58.02	64.32
	Unfixed	**0.80**	5.18	**56.17**	**63.21**
Chair	Fixed	7.41	11.28	60.73	61.48
	Unfixed	**7.01**	**11.15**	**59.67**	**58.76**
Car	Fixed	3.25	8.86	53.69	58.10
	Unfixed	**3.21**	**8.63**	**53.35**	**53.13**

MMD-CD and MMD-EMD are scaled by 10^3 and 10^2 individually
Bold indicate the best results

Table 6 Compare of generation performance between our method with dynamic transformer backbone and with vanilla transformer backbone on car dataset

Model	MMD(↓)		1-NNA(%↓)	
	CD	EMD	CD	EMD
Dynamic transformer	**3.21**	**8.63**	**53.35**	**53.13**
Vanilla transformer	3.26	8.70	57.95	57.67

Bold indicate the best results

Table 7 Comparison of generation performance between different reconstruction loss in first stage training on car dataset

Loss type	MMD(↓)		1-NNA (%↓)	
	CD	EMD	CD	EMD
CD loss	**3.17**	10.32	54.55	88.64
EMD loss	3.22	8.68	55.11	55.97
CD+EMD loss	3.21	**8.63**	**53.35**	**53.13**

Bold indicate the best results

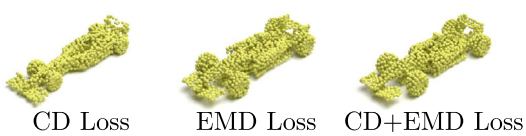

Fig. 8 Visualization of generated point clouds between different reconstruction loss in first stage training on car dataset

latent compressor trained in the first stage. The experimental results are shown in Table 7, and the corresponding visualizations are shown in Fig. 8. It can be seen that using only CD loss performs poorly in terms of EMD distance. This may be because CD loss cannot learn the overall distribution of point clouds well and is not sensitive to changes in point cloud density. On the other hand, using only EMD can generate more outlier points. This may be because EMD is not as sensitive to outliers as CD. Using EMD+CD in the reconstruction loss can effectively combine the advantages of CD and EMD, thereby achieving better generation performance.

5 Conclusion

We proposed a latent diffusion transformer (LDT) for point cloud generation. LDT is a probabilistic, unified, latent diffusion model that can achieve smooth and high-quality point cloud generation without using additional conditions. The effectiveness of our method is attributed to the extraction of surface distribution by the latent compressor and the construction of long-distance dependencies by the dynamic transformer. These modules significantly enhanced the expressiveness of our model, enabling us to effectively learn the global surface distribution of the latent shape. The overall framework resulted in smoother point cloud generations and reduced the presence of outlier points. Experiments showed that our method achieved SOTA unconditional point cloud generation on ShapeNet and multi-modal point cloud completion on ShapeNet-ViPC.

Acknowledgements This work was supported in part by the Beijing Natural Science Foundation (Grant No. 4222020), and in part by the National Natural Science Foundation of China (Grant No. 62241601).

Author Contributions JJ involved in methodology and conceptualization; RZ involved in methodology, writing draft, results analysis, and visualization; ML involved in methodology, writing draft, and results analysis.

Data availability statement The datasets generated during and/or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical and informed consent for data used Not applicable. The current study does not involve humans and animals.

References

1. Aiello, E., Valsesia, D., Magli, E.: Cross-modal learning for image-guided point cloud shape completion. Adv. Neural Inf. Process. Syst. **35**, 37349–37362 (2022)
2. Cai, R., Yang, G., Averbuch-Elor. H., et al.: Learning gradient fields for shape generation. In: European Conference on Computer Vision, pp. 364–381 (2020). https://doi.org/10.1007/978-3-030-58580-8_22
3. Chai, S., Zhuang, L., Yan, F.: Layoutdm: transformer-based diffusion model for layout generation, pp. 18349–18358 (2023). https://doi.org/10.1109/CVPR52729.2023.01760
4. Chang, A.X., Funkhouser, T., Guibas, L. et al.: Shapenet: an information-rich 3d model repository (2015). arXiv preprint arXiv:1512.03012
5. Chang, H., Zhang, H., Jiang, L. et al.: Maskgit: masked generative image transformer, pp. 11305–11315 (2022) https://doi.org/10.1109/CVPR52688.2022.01103

6. Chen, R., Han, S., Xu, J. et al.: Point-based multi-view stereo network. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 1538–1547 (2019). https://doi.org/10.1109/ICCV.2019.00162
7. Chen, Z., Qiu, J., Sheng, B., et al.: Gpsd: generative parking spot detection using multi-clue recovery model. Vis. Comput. **37**(9–11), 2657–2669 (2021). https://doi.org/10.1007/S00371-021-02199-Y
8. Chen, Z., Qiu, G., Li, P., et al.: Mngnas: distilling adaptive combination of multiple searched networks for one-shot neural architecture search. IEEE Trans. Pattern Anal. Mach. Intell. **45**(11), 13489–13508 (2023). https://doi.org/10.1109/TPAMI.2023.3293885
9. Cheng, A.C., Li, X., Liu, S., et al.: Autoregressive 3d shape generation via canonical mapping. In: European Conference on Computer Vision, pp. 89–104 (2022). https://doi.org/10.1007/978-3-031-20062-5_6
10. Cho, J., Zala, A., Bansal, M.: Dall-eval: probing the reasoning skills and social biases of text-to-image generation models. pp 3020–3031 (2023). https://doi.org/10.1109/ICCV51070.2023.00283
11. Dhariwal, P., Nichol, A.: Diffusion models beat gans on image synthesis. Adv. Neural Inf. Process. Syst. **34**, 8780–8794 (2021)
12. Dinh, L., Krueger, D., Bengio, Y.: Nice: Non-linear independent components estimation. In: International Conference on Learning Representations (2015)
13. Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al.: An image is worth 16x16 words: transformers for image recognition at scale. In: International Conference on Learning Representations (2021). https://openreview.net/forum?id=YicbFdNTTy
14. Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al.: An image is worth 16x16 words: transformers for image recognition at scale. In: International Conference on Learning Representations (2021)
15. Groueix, T., Fisher, M., Kim, V.G., et al.: Atlasnet: a papier-mache approach to learning 3d surface generation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp 216–224 (2018) https://doi.org/10.1109/CVPR.2018.00030
16. Harvey, W., Naderiparizi, S., Masrani, V., et al.: Flexible diffusion modeling of long videos. Adv. Neural Inf. Process. Syst. **35**, 27953–27965 (2022)
17. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. Adv. Neural Inf. Process. Syst. **33**, 6840–6851 (2020)
18. Ho, J., Saharia, C., Chan, W., et al.: Cascaded diffusion models for high fidelity image generation. J. Mach. Learn. Res. **23**, 2249–2281 (2022)
19. Huang, R., Lam, M.W., Wang, J. et al.: Fastdiff: a fast conditional diffusion model for high-quality speech synthesis. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, pp. 4157–4163 (2022). https://doi.org/10.24963/ijcai.2022/577
20. Huang, R., Zhao, Z., Liu, H. et al.: Prodiff: Progressive fast diffusion model for high-quality text-to-speech. In: Proceedings of the 30th ACM International Conference on Multimedia, pp. 2595–2605, (2022). https://doi.org/10.1145/3503161.3547855
21. Jiang, N., Sheng, B., Li, P., et al.: Photohelper: portrait photographing guidance via deep feature retrieval and fusion. IEEE Trans. Multimed. **25**, 2226–2238 (2023). https://doi.org/10.1109/TMM.2022.3144890
22. Kim, H., Lee, H., Kang, W.H., et al.: Softflow: probabilistic framework for normalizing flow on manifolds. Adv. Neural Inf. Process. Syst. **33**, 16388–16397 (2020)
23. Kim, J., Yoo, J., Lee, J. et al.: Setvae: learning hierarchical composition for generative modeling of set-structured data. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 15059–15068 (2021). https://doi.org/10.1109/CVPR46437.2021.01481
24. Klokov, R., Boyer, E., Verbeek, J.: Discrete point flow networks for efficient point cloud generation. In: European Conference on Computer Vision, pp. 694–710 (2020). https://doi.org/10.1007/978-3-030-58592-1_41
25. Lai, X., Liu, J., Jiang, L. et al.: Stratified transformer for 3d point cloud segmentation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8500–8509 (2022). https://doi.org/10.1109/CVPR52688.2022.00831
26. Lee, J., Lee, Y., Kim, J. et al.: Set transformer: a framework for attention-based permutation-invariant neural networks. In: International Conference on Machine Learning, pp. 3744–3753 (2019)
27. Li, J., Chen, J., Sheng, B., et al.: Automatic detection and classification system of domestic waste via multimodel cascaded convolutional neural network. IEEE Trans. Ind. Inf. **18**(1), 163–173 (2022). https://doi.org/10.1109/TII.2021.3085669
28. Lin, X., Sun, S., Huang, W., et al.: Eapt: efficient attention pyramid transformer for image processing. IEEE Trans. Multimed. **25**, 50–61 (2023). https://doi.org/10.1109/TMM.2021.3120873
29. Liu, Q., Zhao, J., Cheng, C., et al.: Pointalcr: adversarial latent GAN and contrastive regularization for point cloud completion. Vis. Comput. **38**(9), 3341–3349 (2022). https://doi.org/10.1007/S00371-022-02550-X
30. Liu, Z., Tang, H., Lin, Y., et al.: Point-voxel cnn for efficient 3d deep learning. Adv. Neural Inf. Process. Syst. **32**, 963–973 (2019)
31. Luo, S., Hu, W.: Diffusion probabilistic models for 3d point cloud generation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2837–2845 (2021). https://doi.org/10.1109/CVPR46437.2021.00286
32. Lyu, Z., Kong, Z., Xu, X. et al.: A conditional point diffusion-refinement paradigm for 3d point cloud completion. In: International Conference on Learning Representations (2022)
33. Ma, B., Liu, Y.S., Han, Z.: Reconstructing surfaces for sparse point clouds with on-surface priors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 6315–6325 (2022). https://doi.org/10.1109/CVPR52688.2022.00621
34. Peebles, W., Xie, S.: Scalable diffusion models with transformers, pp. 4172–4182 (2023). https://doi.org/10.1109/ICCV51070.2023.00387
35. Peng, S., Jiang, C., Liao, Y., et al.: Shape as points: a differentiable poisson solver. Adv. Neural Inf. Process. Syst. **34**, 13032–13044 (2021)
36. Qin, Z., Yin, M., Lin, Z., et al.: Three-view generation based on a single front view image for car. Vis. Comput. **37**(8), 2195–2205 (2021). https://doi.org/10.1007/S00371-020-01979-2
37. Ramasinghe, S., Khan, S., Barnes, N., et al.: Spectral-gans for high-resolution 3d point-cloud generation. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 8169–8176 (2020). https://doi.org/10.1109/IROS45743.2020.9341265
38. Rombach, R., Blattmann, A., Lorenz, D. et al.: High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp 10684–10695 (2022). https://doi.org/10.1109/CVPR52688.2022.01042
39. Ruan, L., Ma, Y., Yang, H. et al.: Mm-diffusion: learning multi-modal diffusion models for joint audio and video generation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10219–10228 (2023)
40. Sheng, B., Andge Riaz Ali, P.L., Chen, C.L.P.: Improving video temporal consistency via broad learning system. IEEE Trans. Cybern. **52**(7), 6662–6675 (2022). https://doi.org/10.1109/TCYB.2021.3079311
41. Shu, D.W., Park, S.W., Kwon, U.: 3d point cloud generative adversarial network based on tree structured graph convolutions. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 3859–3868 (2019). https://doi.org/10.1109/ICCV.2019.00396

42. Tchapmi, L.P., Kosaraju, V., Rezatofighi, H. et al.: Topnet: structural point cloud decoder. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 383–392, (2019). https://doi.org/10.1109/CVPR.2019.00047
43. Vahdat, A., Kreis, K., Kautz, J.: Score-based generative modeling in latent space. Adv. Neural Inf. Process. Syst. **34**, 11287–11302 (2021)
44. Wu, J., Zhang, C., Xue, T., et al.: Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling. Adv. Neural Inf. Process. Syst. **29**, 82–90 (2016)
45. Xiang, M., Ye, H., Yang, B., et al.: Multi-space and detail-supplemented attention network for point cloud completion. Appl. Intell. **53**(12), 14971–14985 (2023). https://doi.org/10.1007/s10489-022-04219-3
46. Xiang, P., Wen, X., Liu, Y.S. et al.: Snowflakenet: point cloud completion by snowflake point deconvolution with skip-transformer. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 5499–5509 (2021). https://doi.org/10.1109/ICCV48922.2021.00545
47. Xu, F., Wang, Z., Wang, H., et al.: Dynamic vehicle pose estimation and tracking based on motion feedback for lidars. Appl. Intell. **53**(2), 2362–2390 (2023). https://doi.org/10.1007/s10489-022-03576-3
48. Xu, Q., Xu, Z., Philip, J., et al.: Point-nerf: point-based neural radiance fields. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5438–5448 (2022). https://doi.org/10.1109/CVPR52688.2022.00536
49. Yang, G., Huang, X., Hao, Z. et al.: Pointflow: 3d point cloud generation with continuous normalizing flows. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 4541–4550 (2019). https://doi.org/10.1109/ICCV.2019.00464
50. Yang, Y., Feng, C., Shen, Y. et al.: Foldingnet: point cloud auto-encoder via deep grid deformation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 206–215, (2018). https://doi.org/10.1109/CVPR.2018.00029
51. Yu, X., Rao, Y., Wang, Z., et al.: Pointr: diverse point cloud completion with geometry-aware transformers. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 12498–12507 (2021). https://doi.org/10.1109/ICCV48922.2021.01227
52. Yuan, W., Khot, T., Held, D. et al.: Pcn: point completion network. In: International Conference on 3D Vision (3DV), pp. 728–737 (2018). https://doi.org/10.1109/3DV.2018.00088
53. Zeng, X., Vahdat, A., Williams, F., et al.: Lion: latent point diffusion models for 3d shape generation. Adv. Neural Inf. Process. Syst. **35**, 10021–10039 (2022)
54. Zhang, B., Gu, S., Zhang, B. et al.: Styleswin: transformer-based gan for high-resolution image generation, pp. 11294–11304 (2022). https://doi.org/10.1109/CVPR52688.2022.01102
55. Zhang, X., Feng, Y., Li, S., et al.: View-guided point cloud completion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp 15890–15899 (2021). https://doi.org/10.1109/CVPR46437.2021.01563
56. Zhou, L., Du, Y., Wu, J.: 3d shape generation and completion through point-voxel diffusion. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 5826–5835 (2021). https://doi.org/10.1109/ICCV48922.2021.00577
57. Zhu, Z., Nan, L., Xie, H., et al.: Csdn: cross-modal shape-transfer dual-refinement network for point cloud completion. IEEE Trans. Vis. Comput. Gr. (2023). https://doi.org/10.1109/TVCG.2023.3236061

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Junzhong Ji is currently a Professor and the PhD Supervisor with the Faculty of Information Technology, Beijing University of Technology, Beijing, China, and Youth Skeleton Teacher in Beijing and Senior Member of the China Computer Federation, Beijing. He has authored or coauthored more than 150 articles in these areas. His research interests include data mining, machine learning, swarm intelligence, bioinformatics and brain image intelligence analysis.

Runfeng Zhao received the bachelor degree from the Beijing University of Technology in 2020. He is currently pursuing a master's degree in computer science at the Beijing University of Technology in China. His main research interests include computer vision, 3D synthesis, and 3D reconstruction.

Minglong Lei received the PhD degree in computer science from the University of Chinese Academy of Sciences, Beijing, China, in 2019. He is currently an Associate Professor at the Beijing University of Technology, Beijing, China. His research interests include machine learning and graph data mining, geometric deep learning and temporal data analysis, and processing.